Generative Adversarial Network

Jungtaek Kim (jtkim@postech.ac.kr)

Machine Learning Group,
Department of Computer Science and Engineering, POSTECH,
77 Cheongam-ro, Nam-gu, Pohang 37673,
Gyeongsangbuk-do, Republic of Korea

February 28, 2019

Table of Contents

Generative Adversarial Network

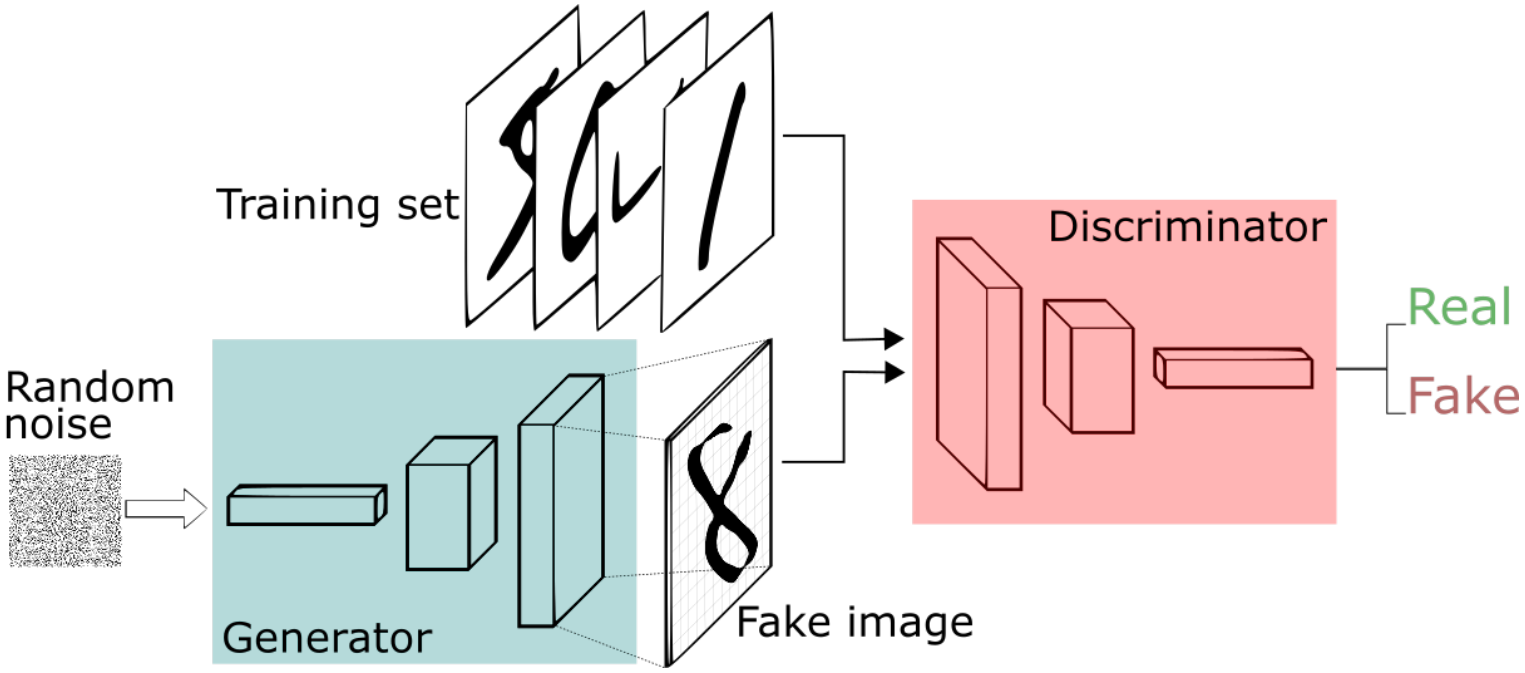
Generative Adversarial Network

Generative Adversarial Network

- A generative model estimated via an adversarial process.
- It trains a generative model G and a discriminative model D simultaneously.
- The training procedure for G is to maximize the probability of D making a mistake.
- It is a minimax two-player game.

- `https://papers.nips.cc/paper/5423-generative-adversarial-nets.pdf`

Generative Adversarial Network



Generative Adversarial Network

- Two-player minimax game with value function $V(G, D)$ is

$$\min_G \max_D V(D, G)$$
$$= \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{x}}(\mathbf{z})}[\log(1 - D(G(\mathbf{x})))]. \quad (1)$$

- For a practical reason, rather than minimizing $\log(1 - D(G(\mathbf{x})))$, we can maximize $\log(D(G(\mathbf{x})))$.

Generative Adversarial Network

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

end for

- Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Generative Adversarial Network

Variations of Generative Adversarial Network

- DCGAN (https://arxiv.org/pdf/1511.06434.pdf)
- Wasserstein GAN (https://arxiv.org/pdf/1701.07875.pdf)
- BigGAN (https://arxiv.org/abs/1809.11096)

DCGAN

Architecture guidelines for stable Deep Convolutional GANs

- Replace any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
- Use batchnorm in both the generator and the discriminator.
- Remove fully connected hidden layers for deeper architectures.
- Use ReLU activation in generator for all layers except for the output, which uses Tanh.
- Use LeakyReLU activation in the discriminator for all layers.

Thank you.